Python & Data Science

How Python Became the Standard

Programming Fundamentals Team

SETU

2026-06-03

Outline

Outline

1. The Perfect Storm (2005-2012)

The convergence of technical and historical factors that made Python dominant in data science

Key ingredients:

- Technical tools became available
- Business demand for data analysis exploded
- Perfect timing with academic adoption
- Community-driven open source development

Outline

2.1 NumPy (2005)

Fast, efficient numerical arrays created by Travis Oliphant

- Filled the gap pure Python couldn't handle
- Made Python competitive with MATLAB and R
- Enabled high-performance numerical computing

2.2 SciPy (2001) & Matplotlib (2003)

- **SciPy**: Scientific computing algorithms
- **Matplotlib**: Easy data visualization and plotting
- Established Python as viable for scientific work

Outline

3.1 Financial Crisis (2008)

- Banks needed better data analysis
- Risk modeling required massive computational power
- Python + NumPy could handle it efficiently

3.2 Web 2.0 & Big Data

- Companies like Google, Facebook, Amazon generated enormous datasets
- Data-driven decision making became competitive advantage
- Need for tools to process massive amounts of data

3.3 Machine Learning Boom

Scikit-learn (2007) released:

- Easy-to-use machine learning algorithms
- Made ML accessible to non-experts
- Simple, consistent API

Outline

4. Pandas: The Game Changer (2008)

Created by: Wes McKinney (AQR Capital)

Why revolutionary:

- DataFrames (table-like data)
- SQL-like operations in Python
- Handle missing data easily
- Reshape and merge datasets

Before Pandas: Manual loops, complex data structures

After Pandas:

```
df = pd.read_csv('data.csv')
result = df[df['age'] > 30]\
  .groupby('dept').sum()
result.plot()
```

Outline

5.1 Universities Switch to Python

- Replaced expensive MATLAB licenses with free Python
- New generation learned Python in school
- Entered industry already knowing the language

5.2 Jupyter Notebooks (2014)

IPython evolved into Jupyter:

- Interactive exploratory data analysis
- Code, results, and documentation in one place
- Perfect for sharing and reproducible research
- Scientists could document entire analyses

Outline

6.1 Deep Learning Libraries (2015-2020)

TensorFlow (2015)

- Google's deep learning framework
- Written with Python-first design

PyTorch (2016)

- Facebook's alternative
- Even more Pythonic and flexible

Keras (2015) — High-level API on top of TensorFlow

6.2 Virtuous Cycle

- ML papers came with Python code
- New researchers learned by reading papers
- Adoption drove more adoption

6.3 Comparison with Alternatives

6.3.1 Python vs R

- Python: General-purpose language
- R: Statistical language only
- Winner: Python (broader ecosystem)

6.4 Python vs MATLAB

- MATLAB: Expensive, proprietary
- Python: Free and open source
- Winner: Python (accessibility)

6.5 Python vs Java/C++

- Traditional languages: Verbose, compilation needed
- Python: Quick prototyping, simple syntax
- Winner: Python (data scientists want simplicity)

Outline

7.1 Timeline of Dominance

Year	Event
2005	NumPy released
2008	Pandas created
2010	Jupyter notebooks appear
2012	Python clearly winning
2015	TensorFlow/PyTorch released
2017	Python 1 for data science
2020	Python most popular overall

7.2 Key Cultural Factor: Open Source

Python's Strength:

- Hundreds of community contributions
- Clear, consistent APIs
- Easy to install (pip)
- Libraries built on libraries

Why R Struggled:

- Many packages, less standardization
- Different conventions between packages
- Harder to chain operations

7.3 Why This Matters Today

7.3.1 Network Effects

```
More people use Python
  ↓
More libraries exist
  ↓
More people use Python (repeat)
```

- Job market wants Python talent
- Universities teach Python
- New scientists learn Python
- The snowball keeps rolling

Outline

8. The Perfect Requirements for Data Science

Python provided everything needed:

✓ Fast computation (NumPy)

✓ Easy data manipulation (Pandas)

✓ Beautiful visualizations (Matplotlib)

✓ Machine learning (Scikit-learn, TensorFlow)

✓ Easy to learn syntax

✓ Reproducible research (Jupyter)

✓ Free and open source

✓ General-purpose language

No other language had all of these at the right time.

8.1 The NumPy Performance Secret

Many people think Python is “too slow” for data science.

The secret: **NumPy operations are not in Python!**

Looks like Python:

```
result = array1 + array2
```

Actually runs: Optimized C and Fortran code

This allowed Python to maintain ease-of-use while achieving competitive performance.

8.2 What if Python Hadn't Won?

Data science would likely:

- Still use R for statistics, MATLAB for computation
- Lack unified ecosystem
- Require multiple languages per project
- Have slower reproducibility
- AI/ML explosion would be delayed

Python's victory was about timing, free access, and simple syntax for domain experts.

8.3 Key Takeaways

1. **Timing matters** — Tools appeared when demand exploded
2. **Community-driven** — Individuals, not top-down mandates
3. **Ecosystem effects** — Libraries built on libraries
4. **Accessibility** — Free, easy syntax, no compilation
5. **Versatility** — Not just for data science
6. **Education** — Universities teaching Python created talent pipeline

8.4 The Perfect Storm

1. Technical Foundations (NumPy, Pandas)
2. Business Demand (Big Data, ML boom)
3. Academic Adoption (Universities)
4. Community Contributions (Open Source)
5. **Python as Data Science Standard**

Thanks for Watching - Any questions?